

Mystery Room — Team 4

Frontend Tasks Division + Second Mystery Backend Guide

Team Leader: Hamid Al Haj

Backend: Hamid + Ali

Frontend: Shiam + Hassan + Mulham + Adham

1. Goal of This Document

This document defines the frontend task division for **Mystery Room — Team 4**.

It explains:

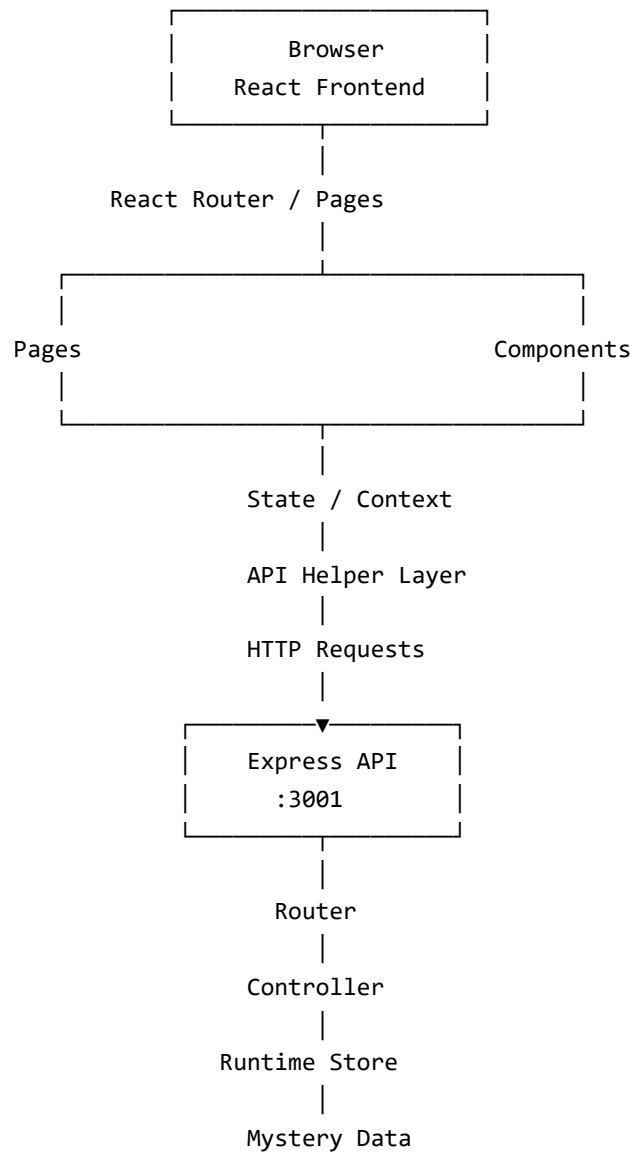
- Who is responsible for which files.
- Which Pages and Components each member should build.
- Which Routes are required.
- How the frontend communicates with the backend.
- Where to use `useState`, Context, or Redux.
- How the game flows from Home to Result.
- How to add a **second Mystery** to the backend using the same architecture.

The goal is not to copy B4F Hub. Instead, we use a similar project organization:

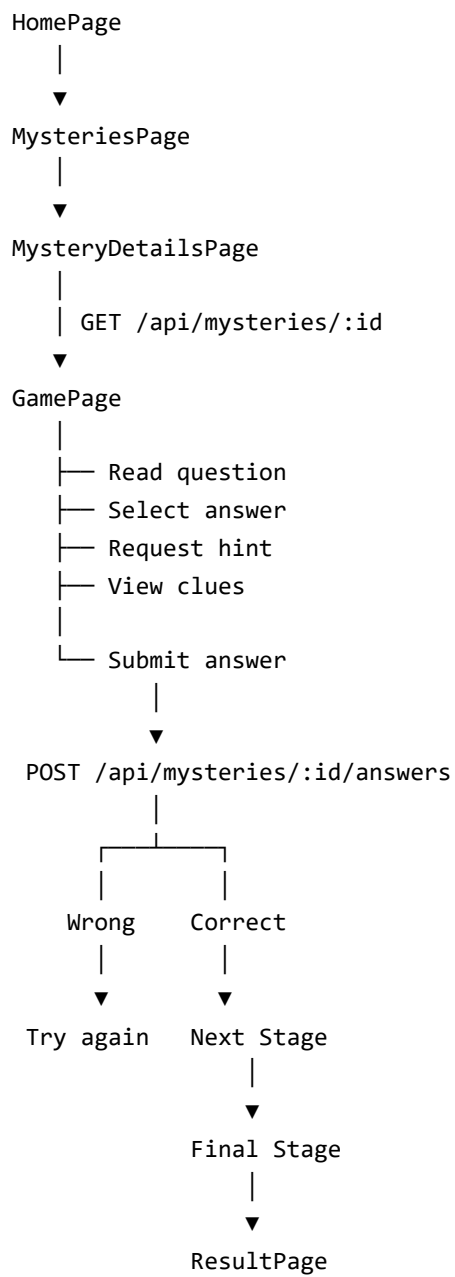
```
pages/  
components/  
api/  
types/  
context/  
store/
```

while keeping the architecture specific to **Mystery Room**.

2. Project Architecture



3. User Flow



4. Frontend Team Division

Member	Main Responsibility	Main Files
Shiam	Pages + Router + Navigation	pages/ , router/ , Header , Navbar , Footer
Hassan	API Layer + Types + Loading/Error	api/ , types/ , LoadingMessage , ErrorMessage
Mulham	Shared State + Context/Redux	context/ , store/ , hooks/selectors
Adham	Game UI Components + Result UI	components/game/ , ResultCard , AnswerOptions , HintBox

Each member should work primarily on their assigned area, but integration is a team responsibility.

5. SHIAM — Pages + Router + Navigation

Responsibility

Shiam is responsible for the application's page structure and navigation.

The objective is to create a real multi-page React application using React Router rather than keeping the entire game inside one component.

Files

```
client/src/
├── pages/
│   ├── HomePage.tsx
│   ├── MysteriesPage.tsx
│   ├── MysteryDetailsPage.tsx
│   ├── GamePage.tsx
│   ├── ResultPage.tsx
│   ├── AboutPage.tsx
│   └── NotFoundPage.tsx
├── router/
│   └── AppRouter.tsx
└── components/
    ├── Header.tsx
    ├── Navbar.tsx
    └── Footer.tsx
```

Required Routes

<code>/</code>	→ <code>HomePage</code>
<code>/mysteries</code>	→ <code>MysteriesPage</code>
<code>/mysteries/:id</code>	→ <code>MysteryDetailsPage</code>
<code>/mysteries/:id/play</code>	→ <code>GamePage</code>
<code>/result/:id</code>	→ <code>ResultPage</code>
<code>/about</code>	→ <code>AboutPage</code>
<code>*</code>	→ <code>NotFoundPage</code>

The `:id` route is important because the same page should be able to handle different mysteries.

For example:

```
/mysteries/1
/mysteries/2
/mysteries/3
```

all use:

```
MysteryDetailsPage
```

AppRouter.tsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

import HomePage from "../pages/HomePage";
import MysteriesPage from "../pages/MysteriesPage";
import MysteryDetailsPage from "../pages/MysteryDetailsPage";
import GamePage from "../pages/GamePage";
import ResultPage from "../pages/ResultPage";
import AboutPage from "../pages/AboutPage";
import NotFoundPage from "../pages/NotFoundPage";

export default function AppRouter() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<HomePage />} />

        <Route
          path="/mysteries"
          element={<MysteriesPage />}
        />

        <Route
          path="/mysteries/:id"
          element={<MysteryDetailsPage />}
        />

        <Route
          path="/mysteries/:id/play"
          element={<GamePage />}
        />

        <Route
          path="/result/:id"
          element={<ResultPage />}
        />

        <Route
          path="/about"
          element={<AboutPage />}
        />

        <Route
          path="*"
          element={<NotFoundPage />}
        />
      </Routes>
    </BrowserRouter>
  );
}
```

Using `useParams`

Inside `MysteryDetailsPage` or `GamePage` :

```
import { useParams } from "react-router-dom";

export default function GamePage() {
  const { id } = useParams();

  return <h1>Mystery ID: {id}</h1>;
}
```

Navigation

Use React Router's `Link` and `NavLink` .

```
<Link to="/mysteries">
  Explore Mysteries
</Link>
```

For navigation after the game:

```
const navigate = useNavigate();

navigate(`/result/${id}`);
```

Avoid using normal `<a href="">` for internal React navigation.

GamePage Structure

`GamePage` should not become a huge file.

Its job is to combine the components created by Adham:

```
GamePage
├─ StageHeader
├─ StageProgress
├─ StageQuestion
├─ AnswerOptions
├─ SubmitAnswerButton
├─ HintButton
├─ HintBox
└─ CluePanel
```

Shiam Checklist

- ☐ HomePage
 - ☐ MysteriesPage
 - ☐ MysteryDetailsPage
 - ☐ GamePage
 - ☐ ResultPage
 - ☐ AboutPage
 - ☐ NotFoundPage
 - ☐ AppRouter
 - ☐ Dynamic route
 - ☐ Navbar
 - ☐ Navigation links
 - ☐ Back navigation
-

6. HASSAN — API + Types + Loading/Error

Responsibility

Hassan owns the communication layer between React and Express.

The objective is to prevent random `fetch()` calls from being spread throughout the Components.

Instead:

```
Component
  ↓
API Function
  ↓
API Client
  ↓
Express Backend
```

7. API Folder

```
client/src/
├── api/
│   ├── client.ts
│   └── mysteryApi.ts
├── types/
│   ├── mystery.ts
│   ├── stage.ts
│   ├── game.ts
│   └── api.ts
└── components/
    ├── LoadingMessage.tsx
    ├── ErrorMessage.tsx
    └── EmptyState.tsx
```

8. API Client

client.ts

```
const API_BASE_URL = "http://localhost:3001/api";

export async function apiRequest<T>({
  path: string,
  options?: RequestInit
}): Promise<T> {
  const response = await fetch(
    `${API_BASE_URL}${path}`,
    options
  );

  if (!response.ok) {
    const data = await response
      .json()
      .catch(() => null);

    throw new Error(
      data?.error || "Request failed"
    );
  }

  return response.json();
}
```

9. Mystery API

mysteryApi.ts

```
import { apiRequest } from "../client";

import type { Mystery } from "../types/mystery";
import type {
  Clue,
  AnswerResponse,
  HintResponse,
} from "../types/api";

export function getMysteries() {
  return apiRequest<Mystery[]>("/mysteries");
}

export function getMysteryById(id: string) {
  return apiRequest<Mystery>(
    `/mysteries/${id}`
  );
}

export function getClues(id: string) {
  return apiRequest<Clue[]>(
    `/mysteries/${id}/clues`
  );
}

export function submitAnswer(
  id: string,
  answer: string
) {
  return apiRequest<AnswerResponse>(
    `/mysteries/${id}/answers`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ answer }),
    }
  );
}

export function requestHint(id: string) {
  return apiRequest<HintResponse>(
    `/mysteries/${id}/hint`,
    {
      method: "PATCH",
    }
  );
}
```

```
    }  
  );  
}
```

The important idea is that other Components should use:

```
getMysteryById(id)
```

instead of writing:

```
fetch("http://localhost:3001/api/mysteries/1")
```

10. TypeScript Types

mystery.ts

```
export interface Mystery {  
  id: number;  
  slug: string;  
  title: string;  
  intro: string;  
  totalStages: number;  
  currentStage: number;  
  solved: boolean;  
  hintsUsed: number;  
  currentQuestion: string;  
  currentOptions: string[];  
}
```

api.ts

```
export interface Clue {
  stage: number;
  text: string;
}

export interface AnswerResponse {
  correct: boolean;
  message: string;
  nextStage?: number | null;
}

export interface HintResponse {
  hint: string;
  hintsRemaining: number;
}
```

11. Loading State

Every API-based page should clearly communicate when data is being loaded.

```
const [loading, setLoading] = useState(true);

if (loading) {
  return <LoadingMessage />;
}
```

Example:

```
export default function LoadingMessage() {
  return (
    <div className="loading">
      Loading mystery...
    </div>
  );
}
```

12. Error State

```
const [error, setError] = useState("");

if (error) {
  return (
    <ErrorMessage message={error} />
  );
}
```

Component:

```
interface Props {
  message: string;
}

export default function ErrorMessage({
  message,
}: Props) {
  return (
    <div className="error">
      <p>{message}</p>

      <button onClick={() => window.location.reload()}>
        Try Again
      </button>
    </div>
  );
}
```

13. Hassan Checklist

- ☐ api/client.ts
- ☐ api/mysteryApi.ts
- ☐ Mystery types
- ☐ Stage types
- ☐ API response types
- ☐ Loading component
- ☐ Error component
- ☐ Empty state
- ☐ API error handling
- ☐ Test all endpoints
- ☐ Coordinate response shapes with Hamid + Ali

14. MULHAM — Context + Redux + Shared State

Responsibility

Mulham is responsible for shared frontend state.

However, Redux should not be added simply because Redux exists in the B4F Hub project.

Before creating global state, ask:

```
Does more than one distant component  
actually need this state?
```

If no:

```
useState
```

is usually enough.

If yes:

```
Context / Redux
```

may be appropriate.

15. Suggested Structure

```
client/src/  
├── context/  
│   ├── ThemeContext.tsx  
│   └── LanguageContext.tsx  
└── store/  
    ├── store.ts  
    ├── hooks.ts  
    ├── selectors.ts  
    └── gameSlice.ts
```

16. Theme Context

Example:

```
import {
  createContext,
  useContext,
  useState,
} from "react";

const ThemeContext = createContext(null);

export function ThemeProvider({
  children,
}) {
  const [dark, setDark] = useState(true);

  return (
    <ThemeContext.Provider
      value={{ dark, setDark }}
    >
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  return useContext(ThemeContext);
}
```

This allows:

```
Navbar
GamePage
Footer
```

to access the same theme without passing props through multiple components.

17. Language Context

If the team decides to support English/Arabic, a Language Context can manage:

```
language = "en"
language = "ar"
```

For example:

```
const { language, setLanguage } =
  useLanguage();
```

This should only be added if the team actually implements the feature.

18. Redux

If Redux Toolkit is used, keep the state small and meaningful.

Example:

```
interface GameState {  
  selectedAnswer: string;  
  gameStatus:  
    | "idle"  
    | "submitting"  
    | "success"  
    | "error";  
}
```

Possible actions:

```
setSelectedAnswer  
setGameStatus  
resetGame
```

19. What Should NOT Be in Redux?

Do not create a fake copy of backend state:

```
{  
  currentStage: 2,  
  solved: true  
}
```

and then treat Redux as the source of truth.

The backend remains responsible for the real mystery state.

A useful separation is:

URL

↓

Mystery ID

Local State

↓

Selected answer

Context

↓

Theme / Language

Backend

↓

Current stage

Solved state

Hints

Game progress

20. Mulham Checklist

- ☐ Decide what truly needs shared state.
 - ☐ Build Theme Context.
 - ☐ Build Language Context if needed.
 - ☐ Configure Redux Toolkit if needed.
 - ☐ Create selectors/hooks if Redux is used.
 - ☐ Avoid duplicating backend state.
 - ☐ Coordinate with Adham about state usage.
 - ☐ Coordinate with Shiam about providers.
-

21. ADHAM — Game Components + Result UI

Responsibility

Adham owns the main interactive game interface.

Instead of putting all game UI into `GamePage.tsx`, split it into reusable Components.

22. Component Structure

```
client/src/components/  
├─ MysteryCard.tsx  
├─ ProgressBar.tsx  
└─  
    └─ game/  
        ├─ StageHeader.tsx  
        ├─ StageQuestion.tsx  
        ├─ AnswerOptions.tsx  
        ├─ SubmitAnswerButton.tsx  
        ├─ HintButton.tsx  
        ├─ HintBox.tsx  
        ├─ CluePanel.tsx  
        ├─ StageProgress.tsx  
        ├─ GameStatus.tsx  
        ├─ SuccessMessage.tsx  
        ├─ WrongAnswerMessage.tsx  
        └─ ResultCard.tsx
```

23. StageQuestion

```
interface Props {  
  question: string;  
}  
  
export default function StageQuestion({  
  question,  
}: Props) {  
  return <h2>{question}</h2>;  
}
```

24. AnswerOptions

```
interface Props {
  options: string[];
  selected: string;
  onSelect: (answer: string) => void;
}

export default function AnswerOptions({
  options,
  selected,
  onSelect,
}: Props) {
  return (
    <div>
      {options.map((option) => (
        <button
          key={option}
          className={
            selected === option
              ? "selected"
              : ""
          }
          onClick={() => onSelect(option)}
        >
          {option}
        </button>
      ))}
    </div>
  );
}
```

25. Hint Button

The Hint Button should not know how the API works.

It only triggers a callback:

```
interface Props {
  onClick: () => void;
  disabled?: boolean;
}

export default function HintButton({
  onClick,
  disabled,
}: Props) {
  return (
    <button
      onClick={onClick}
      disabled={disabled}
    >
      Show Hint
    </button>
  );
}
```

The API call remains in the API layer.

26. Hint Box

```
interface Props {
  hint: string;
}

export default function HintBox({
  hint,
}: Props) {
  return (
    <div className="hint-box">
      <strong>Hint</strong>
      <p>{hint}</p>
    </div>
  );
}
```

27. Clue Panel

`CluePanel` displays clues received from:

```
GET /api/mysteries/:id/clues
```

It should be possible to show/hide the clues without reloading the page.

Possible structure:

Clues
• The clock stopped at 9:15 • The door was locked • A note was found

28. ResultCard

Example:

✓ Mystery Solved
You found the missing message.
Stages completed: 3 / 3
Hints used: 1
[Play Another Mystery]

29. Adham Checklist

- ☐ StageHeader
- ☐ StageQuestion
- ☐ AnswerOptions
- ☐ SubmitAnswerButton
- ☐ HintButton
- ☐ HintBox
- ☐ CluePanel
- ☐ StageProgress
- ☐ GameStatus
- ☐ SuccessMessage
- ☐ WrongAnswerMessage
- ☐ ResultCard
- ☐ Responsive game UI

30. How GamePage Combines Everything

The final `GamePage` should conceptually look like this:

```
export default function GamePage() {
  const { id } = useParams();

  // API + local/shared state

  return (
    <main>
      <StageHeader />

      <StageProgress />

      <StageQuestion
        question={mystery.currentQuestion}
      />

      <AnswerOptions
        options={mystery.currentOptions}
        selected={selectedAnswer}
        onSelect={setSelectedAnswer}
      />

      <SubmitAnswerButton
        onClick={handleSubmit}
        disabled={
          !selectedAnswer || submitting
        }
      />

      <HintButton
        onClick={handleHint}
      />

      <HintBox hint={hint} />

      <CluePanel clues={clues} />

      <GameStatus status={status} />
    </main>
  );
}
```

The responsibilities are now clear:

Shiam



GamePage + Route

Hassan



API + Types

Mulham



Shared State / Context

Adham



Game Components

31. Git Branch Strategy

Each frontend member should use a separate branch:

```
main
|
├─ feature/shiam-pages-router
├─ feature/hassan-api-types
├─ feature/mulham-state-context
└─ feature/adham-game-components
```

Example commits:

Shiam

feat: add mystery room routes and pages

Hassan

feat: add mystery API helper functions and types

Mulham

feat: add theme context and shared game state

Adham

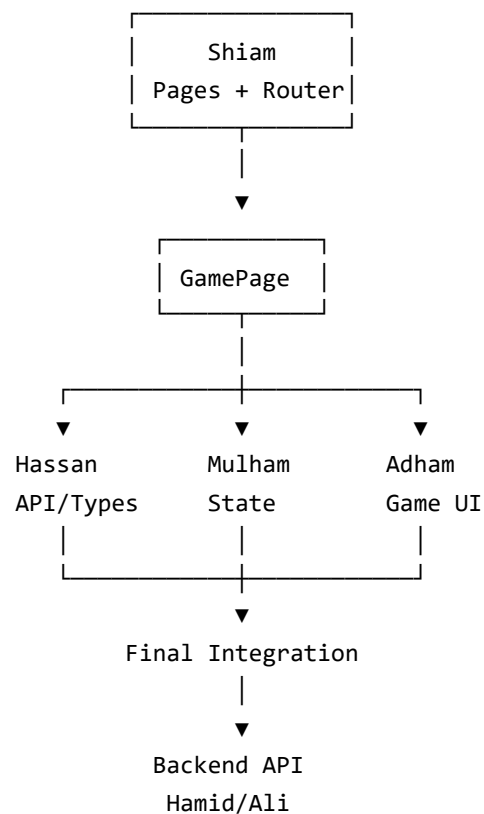
feat: add interactive game components

Hamid / Ali

fix: validate mystery answer submission

Avoid having everyone edit the same large file whenever possible.

32. Integration Workflow



33. Development Order

The team should not wait for one person to finish everything before starting.

Phase 1 — Structure

Shiam

Build:

- Pages
- Router
- Navbar

Hassan

Build:

- API client
- Types
- API functions

Mulham

Build:

- Context
- State architecture

Adham

Build components using temporary props:

```
<StageQuestion  
  question="Sample question"  
>
```

34. Phase 2 — Integration

Then:

Shiam



connects Pages

Hassan



connects real API

Adham



replaces mock data with props

Mulham



connects shared state where necessary

35. Frontend Requirements Matrix

Requirement	Main Owner	Support
React frontend	Everyone	—
3+ routes	Shiam	Everyone
Dynamic route	Shiam	Hassan
API helper layer	Hassan	Hamid/Ali
Loading state	Hassan	Shiam/Adham
Error state	Hassan	Shiam
Answer input	Adham	Mulham
Game screen	Shiam + Adham	Hassan
Result screen	Shiam + Adham	Hassan
Context	Mulham	Shiam
Redux if needed	Mulham	Adham/Hassan
Responsive UI	Adham	Shiam/Hassan
Integration	Hamid	Everyone

36. Second Mystery — Backend

After the first mystery is stable, the team can add a second mystery using the same backend architecture.

The team should choose the story themselves.

Possible directions:

- Historical mystery
- Robbery mystery
- Hacker mystery
- Train mystery
- Abandoned room
- Treasure mystery
- Cybersecurity mystery
- Scientific mystery

The important part is not the category.

The second mystery should have:

- 3 stages
- 3 questions
- 3 hints
- 3 clues

and should be more challenging than the first mystery.

37. Second Mystery Data Structure

The existing mystery should follow the same structure.

Conceptually:

```
export const initialMysteries = [
  {
    id: 1,
    slug: "sealed-room",
    title: "Mystery One",
    intro: "...",
    totalStages: 3,

    stages: [
      {
        id: 0,
        question: "...",
        answer: "...",
        options: ["A", "B", "C", "D"],
        hint: "...",
        clue: "...",
      },

      {
        id: 1,
        question: "...",
        answer: "...",
        options: ["A", "B", "C", "D"],
        hint: "...",
        clue: "...",
      },

      {
        id: 2,
        question: "...",
        answer: "...",
        options: ["A", "B", "C", "D"],
        hint: "...",
        clue: "...",
      },
    ],

    solved: false,
    currentStage: 0,
    hintsUsed: 0,
  },

  {
    id: 2,
    slug: "YOUR-NEW-SLUG",
    title: "YOUR-NEW-TITLE",
    intro: "YOUR-NEW-STORY",
    totalStages: 3,

    stages: [
      {
```

```
    id: 0,
    question: "QUESTION 1",
    answer: "ANSWER 1",
    options: ["A", "B", "C", "D"],
    hint: "HINT 1",
    clue: "CLUE 1",
  },

  {
    id: 1,
    question: "QUESTION 2",
    answer: "ANSWER 2",
    options: ["A", "B", "C", "D"],
    hint: "HINT 2",
    clue: "CLUE 2",
  },

  {
    id: 2,
    question: "QUESTION 3",
    answer: "ANSWER 3",
    options: ["A", "B", "C", "D"],
    hint: "HINT 3",
    clue: "CLUE 3",
  },
],

solved: false,
currentStage: 0,
hintsUsed: 0,
},
];
```

38. Why We Do Not Need New Endpoints

The existing API can already work with different mystery IDs:

```
GET    /api/mysteries
GET    /api/mysteries/:id
GET    /api/mysteries/:id/clues
POST   /api/mysteries/:id/answers
PATCH /api/mysteries/:id/hint
```

Therefore:

```
GET /api/mysteries/1
  ↓
Mystery 1
```

and:

```
GET /api/mysteries/2
  ↓
Mystery 2
```

can use the same Router and Controller.

This is exactly why we use a dynamic `:id`.

39. Frontend Support for Mystery 2

Do **not** create:

```
Mystery1Page.tsx
Mystery2Page.tsx
Mystery3Page.tsx
```

Instead:

```
/mysteries/1/play
/mysteries/2/play
```

both use:

```
GamePage
```

The page simply requests:

```
const { id } = useParams();

const mystery = await getMysteryById(id!);
```

This means the frontend is reusable.

40. Responsibilities for Mystery 2

Hamid + Ali

1. Choose the second mystery concept with the team.
2. Write 3 stages.
3. Make the questions harder than Mystery 1.
4. Add:

- `answer`
- `options`
- `hint`
- `clue`

5. Add the mystery to the backend data.

6. Test:

```
GET /api/mysteries
GET /api/mysteries/2
GET /api/mysteries/2/clues
POST /api/mysteries/2/answers
PATCH /api/mysteries/2/hint
```

Shiam

No `Mystery2Page.tsx` is required.

The existing dynamic routes should automatically support:

```
/mysteries/2
/mysteries/2/play
```

The mystery list should simply display the second mystery.

Hassan

Do not create:

```
getMysteryTwo()
```

Instead use the existing reusable function:

```
getMysteryById("2");
```

This proves that the API layer is reusable.

Mulham

Make sure shared state does not depend on one specific mystery.

Avoid:

```
{  
  mysteryOneSolved: false  
}
```

Prefer generic state when state is actually needed:

```
{  
  selectedAnswer: "",  
  gameStatus: "idle"  
}
```

Adham

All Game Components should work with any data received from the API:

```
question  
options  
hint  
clue  
stage
```

No Component should contain hard-coded Mystery 1 questions.

41. Making Mystery 2 More Difficult

Difficulty does not need to mean difficult general-knowledge questions.

The challenge can come from deduction.

For example:

```
Stage 1  
Pattern recognition
```

↓

```
Stage 2  
Use information from Stage 1
```

↓

```
Stage 3  
Final deduction using previous clues
```

Or:

Stage 1 → Logic
Stage 2 → Observation
Stage 3 → Final deduction

The team should choose the actual puzzle design.

42. Hint System Note

If the current backend only calculates:

```
const hintsRemaining =  
  Math.max(0, 3 - mystery.hintsUsed);
```

that does not necessarily prevent a fourth hint request.

If the team wants a hard maximum of three hints, the controller should explicitly reject further requests.

Example:

```
if (mystery.hintsUsed >= 3) {  
  return res.status(409).json({  
    error: "No hints remaining.",  
  });  
}
```

Then the frontend can disable the button when:

```
hintsRemaining === 0
```

43. Final Testing Flow

Test the first mystery entirely through the UI:

Home
↓
Mysteries
↓
Choose Mystery 1
↓
Details
↓
Play
↓
Wrong Answer
↓
Try Again
↓
Correct Answer
↓
Stage 2
↓
Stage 3
↓
Solved
↓
Result

Then repeat with:

Mystery 2

Also test the backend separately using Postman, Thunder Client, or another API client.

44. Definition of Done

The project is ready when:

Frontend works

+

Backend works

+

Routes work

+

Dynamic Route works

+

API layer exists

+

Loading state exists

+

Error state exists

+

Answer input works

+

Wrong answer works

+

Correct answer works

+

Next stage works

+

Final result works

+

Mystery 1 works

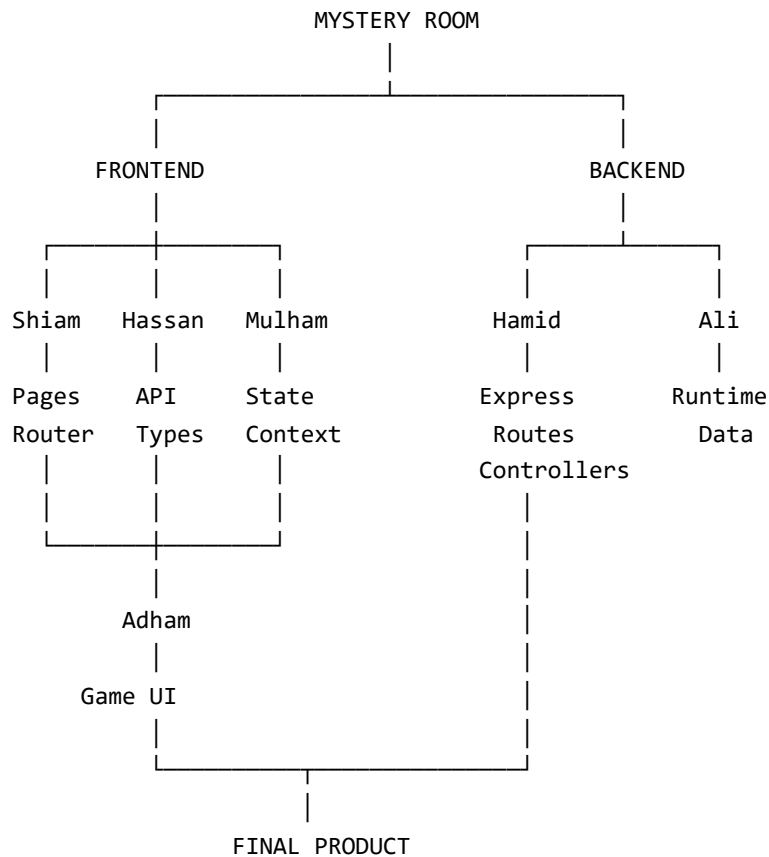
+

Mystery 2 works

+

API endpoints can be tested independently

45. Final Team Structure



Team principle

- Shiam → Where the user goes
- Hassan → How the frontend talks to the backend
- Mulham → What frontend state is shared
- Adham → What the user sees and interacts with
- Hamid → Backend architecture + integration
- Ali → Backend implementation + testing

The objective is to keep the project modular enough that adding a second mystery does **not** require rewriting the frontend.